Experiment - 8

Nikhil Kale
D15A/50

Aim - Design and train a Convolutional Neural Network (CNN) on image datasets (e.g., MNIST/CIFAR)

Dataset Source

Dataset Name: https://github.com/zalandoresearch/fashion-mnist
Source: Kaggle
Title: Fashion-MNIST Dataset

Dataset Description

Fashion-MNIST is a dataset of Zalando fashion product images used for image classification.

- Total images: **70,000**
- Image size: **28 × 28 pixels (grayscale)**
- Classes: **10 fashion categories**

Split:

- Training: 60,000
- Testing: 10,000

Classes include clothing items such as shirts, shoes, bags, and dresses.

Characteristics:

- Balanced dataset
- More complex than MNIST
- Suitable for CNN-based classification

Mathematical Formulation

Convolution Operation

$$Feature\ Map = Input * Kernel + Bias$$

Activation Function (ReLU)

$$f(x) = \max(0, x)$$

Softmax Function

$$P(y_i) = \frac{e^{z_i}}{\sum e^{z_j}}$$

Loss Function

Categorical Cross-Entropy:

$$L = -\sum y_i \log(\hat{y}_i)$$

Algorithm Limitations

- Requires high computational power
- Needs large training data
- Sensitive to hyperparameters
- Overfitting possible
- Hard to interpret

Methodology / Workflow

1. Load dataset
2. Normalize pixel values
3. Reshape images for CNN
4. Encode labels
5. Build CNN model
6. Train using training data
7. Evaluate on test data
8. Analyze results

Workflow:

Dataset → Preprocessing → CNN → Training → Evaluation → Prediction

Performance Analysis

Metrics Used:

- Accuracy
- Precision
- Recall
- F1 Score
- Confusion Matrix

Observations:

- Accuracy ≈ **88–92%**
- Lower than MNIST due to complexity
- Some confusion between similar classes (e.g., shirt vs coat)
- CNN effectively extracts spatial features

Hyperparameter Tuning

Parameters tuned:

- Number of filters (32, 64)
- Kernel size (3×3)
- Batch size (64)
- Epochs (10)
- Dropout (0.3)

Observations:

- More layers → better feature extraction
- Dropout reduces overfitting
- Adam optimizer gives fast convergence

Code and Output -

[1]
10s

```python
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.datasets import fashion_mnist
```

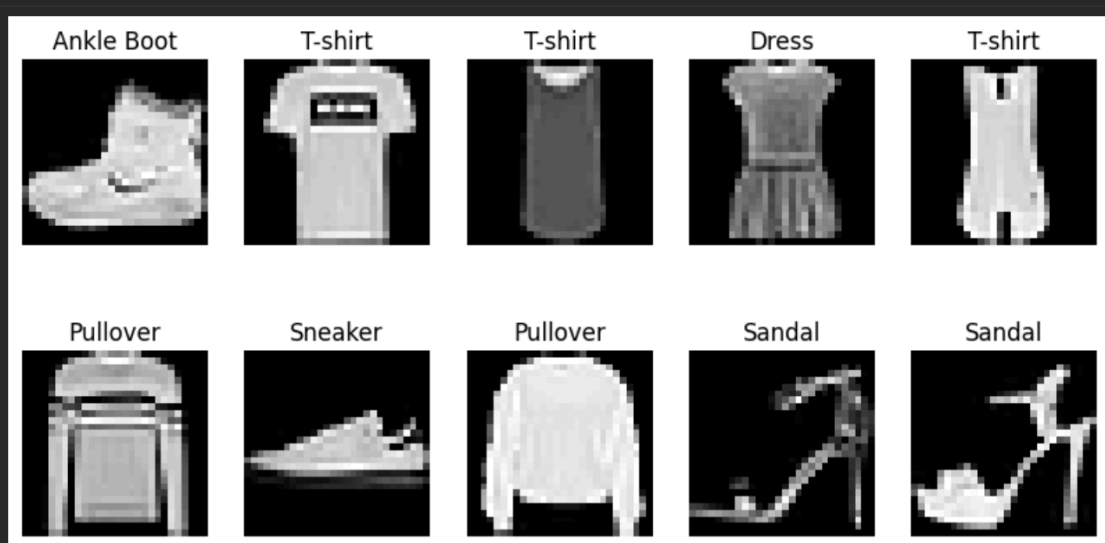
[2]
1s

```python
(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()

print(X_train.shape, X_test.shape)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
(60000, 28, 28) (10000, 28, 28)
```

[3]
1s

```python
class_names = ['T-shirt','Trouser','Pullover','Dress','Coat',
               'Sandal','Shirt','Sneaker','Bag','Ankle Boot']

plt.figure(figsize=(10,5))
for i in range(10):
    plt.subplot(2,5,i+1)
    plt.imshow(X_train[i], cmap='gray')
    plt.title(class_names[y_train[i]])
    plt.axis('off')
plt.show()
```



[4]
0s

```python
X_train = X_train / 255.0
X_test = X_test / 255.0
```

[5]
0s

```python
X_train = X_train.reshape(-1,28,28,1)
X_test = X_test.reshape(-1,28,28,1)
```

[6]
0s

```python
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)
```

[7]
0s

```python
model = Sequential()

# Conv Layer 1
model.add(Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)))
model.add(MaxPooling2D((2,2)))

# Conv Layer 2
model.add(Conv2D(64, (3,3), activation='relu'))
model.add(MaxPooling2D((2,2)))

# Flatten
model.add(Flatten())

# Fully Connected
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.3))

model.add(Dense(10, activation='softmax'))
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

[8]
0s

```python
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

[9]
10m

```python
history = model.fit(
    X_train, y_train,
    epochs=10,
    batch_size=64,
    validation_split=0.2
)
```

```
Epoch 1/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 50s 64ms/step - accuracy: 0.7903 - loss: 0.5743 - val_accuracy: 0.8630 - val_loss: 0.3866
Epoch 2/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 80s 61ms/step - accuracy: 0.8643 - loss: 0.3760 - val_accuracy: 0.8814 - val_loss: 0.3316
Epoch 3/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 83s 62ms/step - accuracy: 0.8826 - loss: 0.3217 - val_accuracy: 0.8928 - val_loss: 0.2916
Epoch 4/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 44s 59ms/step - accuracy: 0.8928 - loss: 0.2908 - val_accuracy: 0.8967 - val_loss: 0.2844
Epoch 5/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 47s 62ms/step - accuracy: 0.9015 - loss: 0.2673 - val_accuracy: 0.8963 - val_loss: 0.2778
Epoch 6/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 81s 62ms/step - accuracy: 0.9074 - loss: 0.2466 - val_accuracy: 0.9094 - val_loss: 0.2546
Epoch 7/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 81s 60ms/step - accuracy: 0.9136 - loss: 0.2307 - val_accuracy: 0.9097 - val_loss: 0.2480
Epoch 8/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 47s 62ms/step - accuracy: 0.9200 - loss: 0.2157 - val_accuracy: 0.9103 - val_loss: 0.2485
Epoch 9/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 44s 59ms/step - accuracy: 0.9247 - loss: 0.2016 - val_accuracy: 0.9092 - val_loss: 0.2584
Epoch 10/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 45s 60ms/step - accuracy: 0.9297 - loss: 0.1869 - val_accuracy: 0.9143 - val_loss: 0.2443
```

[10]
3s

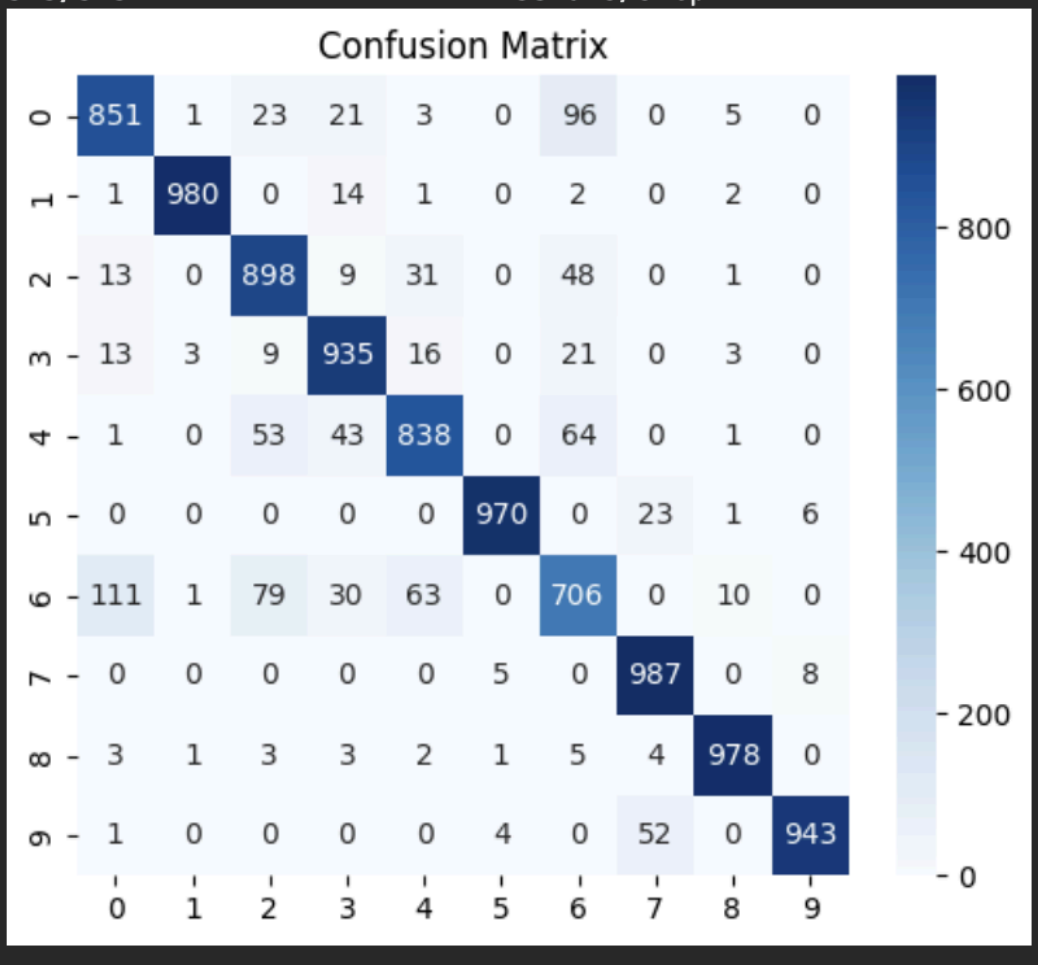
```python
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test Accuracy:", test_acc)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.9086 - loss: 0.2523
Test Accuracy: 0.9085999727249146
```

[11]
✓ 6s

```python
from sklearn.metrics import confusion_matrix
import seaborn as sns

y_pred = model.predict(X_test)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true = np.argmax(y_test, axis=1)

cm = confusion_matrix(y_true, y_pred_classes)

plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix")
plt.show()
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step
```



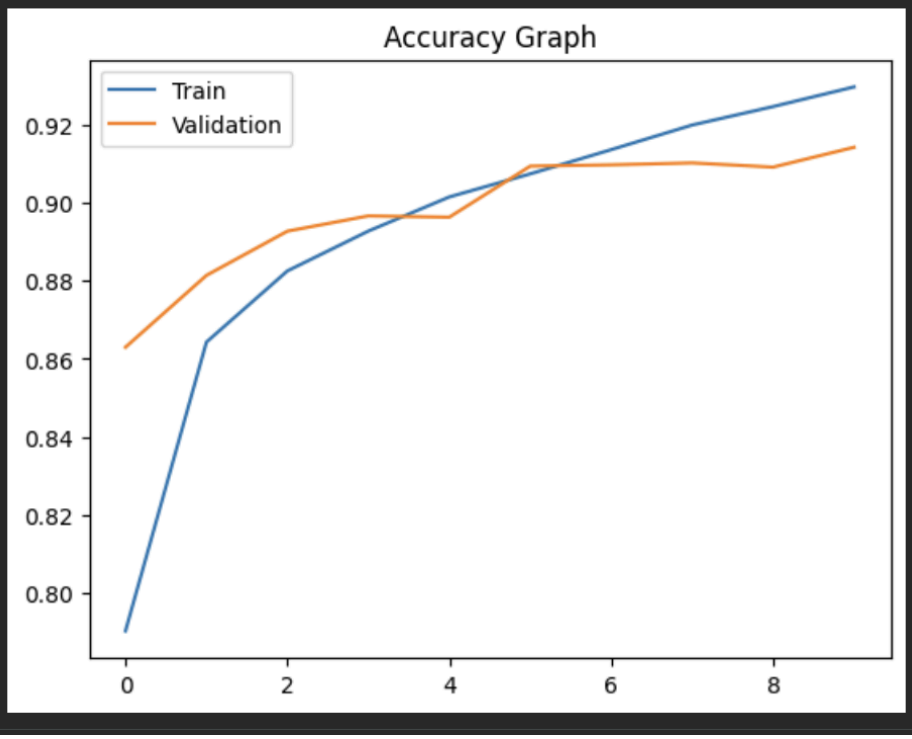
[12]
✓ 0s

```python
from sklearn.metrics import classification_report

print(classification_report(y_true, y_pred_classes))
```

```
              precision    recall  f1-score   support

           0       0.86      0.85      0.85      1000
           1       0.99      0.98      0.99      1000
           2       0.84      0.90      0.87      1000
           3       0.89      0.94      0.91      1000
           4       0.88      0.84      0.86      1000
           5       0.99      0.97      0.98      1000
           6       0.75      0.71      0.73      1000
           7       0.93      0.99      0.96      1000
           8       0.98      0.98      0.98      1000
           9       0.99      0.94      0.96      1000

    accuracy                           0.91     10000
   macro avg       0.91      0.91      0.91     10000
weighted avg       0.91      0.91      0.91     10000
```

[13]
✓ 0s

```python
plt.plot(history.history['accuracy'], label='Train')
plt.plot(history.history['val_accuracy'], label='Validation')
plt.legend()
plt.title("Accuracy Graph")
plt.show()
```



Conclusion

In this experiment, a Convolutional Neural Network (CNN) was successfully implemented on the Fashion-MNIST dataset.

The model achieved good accuracy (~90%) and demonstrated strong capability in classifying real-world images. Compared to MNIST, this dataset presents a more challenging problem, highlighting the effectiveness of CNNs in extracting complex visual features.